

`std::span` and the missing constructor

Document number:	P2447R2
Date:	2022-01-21
Project:	Programming Language C++, Library Working Group
Reply-to:	federico.kircheis@gmail.com

1. Motivation

1.1. A simpler API

Given the functions signature `void foo(std::span<const int>;`, how to call `foo` with a constant set of values?

before	after
<pre>foo({});</pre> CPP	<pre>foo({});</pre> CPP
<pre>foo({1,2,3});</pre> CPP <pre>// compiler error // could not convert '{1,2,3}' from '<brace- enclosed initializer list>' to 'std::span<const int>'</pre>	<pre>foo({1,2,3});</pre> CPP
<pre>int data[]{1,2,3}; foo(data);</pre> CPP	<pre>foo({1,2,3});</pre> CPP
<pre>foo(std::vector<int>{1,2,3});</pre> CPP	<pre>foo({1,2,3});</pre> CPP
<pre>foo(std::initializer_list<int>{1,2,3});</pre> CPP	<pre>foo({1,2,3});</pre> CPP
<pre>foo({{1,2,3}});</pre> CPP	<pre>foo({1,2,3});</pre> CPP
<pre>template<class T> using raw_array = T[]; foo(raw_array<int>{1,2,3});</pre> CPP	<pre>foo({1,2,3});</pre> CPP
<pre>using int_array = int[]; foo(int_array{1,2,3});</pre> CPP	<pre>foo({1,2,3});</pre> CPP

<pre>foo(std::array({1,2,3}));</pre>	CPP	<pre>foo({1,2,3});</pre>	CPP
<pre><i>/* define somewhere a helper function as_span that takes an initializer_list and returns a span */ foo(as_span({1,2,3}));</i></pre>	CPP	<pre>foo({1,2,3});</pre>	CPP
<pre><i>/* define an overload for foo somewhere */ void foo(std::initializer_list<int> il){ return foo(std::span<int>(il)); } foo({1,2,3});</i></pre>	CPP	<pre>foo({1,2,3});</pre>	CPP

The fact that it is not possible in an easy and natural way to create a span over a set of given values is the main motivation for this paper.

Following pattern:

<pre>foo({1,2,3});</pre>	CPP
--------------------------	-----

does currently not compile, as `std::span` does not have a constructor that takes a `std::initializer_list`.

Note that in the case of an empty container, `foo({})` compiles making the construction of span apparently inconsistent.

The after/before table shows different types of workarounds on how to handle this situation.

The most trivial workaround is creating an array and pass it to `foo`. This introduces a variable that was not there before, which also has a much bigger scope (which might or might be not an issue), thus this approach has a clear drawback.

Notice that one could also use a local `std::initializer_list`, and `std::span` range constructor will accept it without issues.

Another approach is creating a temporary container, the easiest one to write is `std::vector`, but doing so introduces unnecessary overhead.

One could use a temporary array, which is more subtle than it needs to be, or `std::array`.

Another alternative is writing a helper function, that takes a set of values and converts them to a span.

When using `std::array`, the syntax between empty and non-empty `std::array` is inconsistent, unless one does not take advantage of class template argument deduction, as `foo(std::array{1,2,3})` compiles, but `foo(std::array{})` does not. Writing the size by hand is not really user-friendly, but at least would make the code consistent, as both `foo(std::array<int,3>{1,2,3});` and `foo(std::array<int,0>{});` compiles.

Last but not least, one can write `foo({{1,2,3}})`, which picks the array overload. This is probably the best/less verbose approach, but the syntax is uncommon, the reasons which overload gets picked unclear and it is also easy to oversee the second pair of brackets. Unfortunately this syntax is inconsistent when dealing with empty spans, as `foo({{}})` does not compile.

Spelling `std::initializer_list<int>` out makes the code compile. As normally one would not write `std::initializer_list` when initializing a container, it makes span construction awkward.

To sum it up:

- in all cases, to call `foo` with a given set of values, it is necessary to manually add some sort of indirection.
- the less verbose approach is the less obvious
- the simplest approaches (temporary vector, local container) have drawbacks
- all approaches, except using a helper function, spelling `std::initializer_list` out or using `std::array` without type deduction, are inconsistent between empty and non-empty spans.

This makes `std::span` more difficult to use than expected.

1.2. Upgrading from vector and vocabulary type

While the main motivation for adding another constructor to span has already been presented, another motivation that led to the creation of this paper is the introduction of `std::span` in existing code.

Consider the following function signature:

```
void foo(const std::vector<int>&);
```

CPP

As the parameter is passed by const-reference, and `foo` does not need the ownership of the data, using `std::vector` is often not optimal especially since we have `std::span`.

"upgrading"/"enhancing" the function signature to

```
void foo(std::span<const int>);
```

CPP

is a code incompatible change for the user of such function. Even recompiling the whole codebase (thus ignoring ABI issues) can lead to compiler errors.

Even if there are trivial code-transformations for fixing the signature incompatibility it is harder to introduce `std::span` in a bigger and older codebase.

The [Usability Enhancements for std::span](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1024r3.pdf) (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1024r3.pdf) paper, gives another reason why we should have a constructor for `std::initializer_list` (emphasis added):

“A span is [...]. It is intended as a new "vocabulary type" for contiguous ranges, replacing the use of(pointer, length) pairs and, in some cases, **vector<T, A>& function parameters**.

— Usability Enhancements for `std::span`

This can be also read from the original [span paper](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0122r1.pdf) (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0122r1.pdf):

“To simplify use of span as a simple parameter, span offers a number of constructors for common container types that store contiguous sequences of elements.

Upgrading `void foo(const std::vector<int>&);` to `void foo(std::span<const int>);` is one of the main use-cases of `std::span`.

The transition from one function signature to another should be as smooth and error-free as possible.

The current incompatibility adds unnecessary work to the users of the function. Making such a change might thus not be possible, especially for a public API.

1.2.1. Does that imply that `std::span` should have all constructors of `std::vector` (or other container with sequential storage)?

No.

While one could write

```
void f(const std::vector<std::string>&);  
foo({4, "hello"}); // vector with 4 times the string "hello"
```

CPP

if `span` would support `std::initializer_list` such code would still not compile.

Nevertheless, based on personal experience, most temporary vectors (or other containers) are either constructed from a constant set of values (thus with a `std::initializer_list`), or empty.

Thus while it is true that even if this paper gets accepted the upgrade process from `std::vector` to `std::span` is still a breaking change, the chances of breaking existing code are much lower.

2. Why doesn't `span` have a `std::initializer_list` constructor?

As a matter of fact, it is possible to create a `span` from an `std::initializer_list` by spelling the type out.

The absence of such constructor seems to imply that a `span` should not be constructed from an `std::initializer_list` (as normally one does not spell `std::initializer_list` out).

I could not find any rationale for not adding such constructor, except those presented in this issue:

<https://github.com/Microsoft/GSL/issues/459>

2.1. dangling spans

The first argument is that if we added such a constructor, it would be easier to create a dangling `span` accidentally:

```
std::span<const int> sp = {1, 2, 3}; // dangles immediately  
std::span<const int> sp{1, 2, 3}; // dangles immediately
```

CPP

However, this argument is weak, because it is already possible to create a dangling `span` variable in many ways. For example:

```
// ---
std::span<const int> sp = {{1, 2, 3}}; // dangles immediately
std::span<const int> sp{{1, 2, 3}}; // dangles immediately

// ---
std::span<const int> sp = std::vector<int>{1,2,3}; // dangles immediately

// ---
// suppose that std::span<const int> bar(); changed to
std::vector<int> bar();

std::span<const int> sp = bar(); // dangles immediately after signature change of bar

// ---
std::span<const int> sp;
{
    const auto v = std::vector<int>{1,2,3};
    sp = v;
}
// dangling sp
```

rvalues are not necessarily short-lived, and the lifetime of an lvalue might be shorter than the lifetime of the constructed span.

Thus using value categories for determining the lifetime, as in this case (if the motivation found on GitHub is the one why span does not have the proposed constructor), gives a false sense of security, and disallows valid use-cases.

`std::span` behaves like a pointer and an associated size, as it does not own the resource.

Thus dangling spans are unavoidable, just like dangling references and pointers. Fortunately, it is possible to diagnose those types of errors statically (<https://godbolt.org/z/Kxocq5qqv>) without making span harder to use.

Also note that `std::string_view`, which has been designed similarly to span to be a type mainly used as "parameter type" for functions, can be constructed from rvalues. Nonetheless being able to create a `std::string_view` from a temporary `std::string` has been an explicit decision (just like creating a span from other rvalues is possible).

2.2. span is a view over a container

The other presented argument is that span is a view over a container and that `std::initializer_list` is not a container.

A `std::span` can be constructed from "something" that is contiguous memory (`std::ranges::data` and `std::ranges::size` needs to work on it). That "something" does thus not even need to be a container.

Also, a pointer and a length are not a container, yet there is a constructor for it, as this is another main use-case for span.

`std::initializer_list` might not be defined as a container, but

- it has contiguous memory
- it has a container-like interface (member functions `begin`, `end` and `size`, not even an `array` has those. It also has free functions for `cbegin`, `rbegin`, ...)
- `std::span` can be already constructed from a `std::initializer_list`, one must "just" be very explicit about it when writing it in code.

As it is possible to create a span from a `std::initializer_list`, and as `std::initializer_list` can be used like a container (just like span), the distinction for span between a `std::initializer_list` and other containers is artificial.

Also adding the proposed constructor does not change in any way what span is or can do.

3. Design Decisions

This is purely a library extension.

It is sufficient to add a constructor for `std::initializer_list` to `std::span`.

Similar to the constructor that takes an array, it is unconditionally `noexcept`, as none of the operations for creating a `std::span` over a `std::initializer_list` can fail.

Similarly to other constructors the proposed constructor is `explicit` if `extent != dynamic_extent`.

As `std::initializer_list` provides only constant access to the elements, this constructor is only available for a span over constant elements.

A reference implementation provided by Arthur O'Dwyer for libc++ can be found on [on github](https://github.com/Quuxplusone/llvm-project/commit/span-ctor) (<https://github.com/Quuxplusone/llvm-project/commit/span-ctor>), he also provided a [playground on godbolt](https://p1144.godbolt.org/z/aKz3PvMKP) (<https://p1144.godbolt.org/z/aKz3PvMKP>).

The proposed changes would also standardize existing practice. The [abseil library](https://github.com/abseil/abseil-cpp/blob/master/absl/types/span.h) (<https://github.com/abseil/abseil-cpp/blob/master/absl/types/span.h>) has `absl::Span`, which works similarly to `std::span` but accepts an `std::initializer_list` since it's first commit on Github made in 2017, with following motivation:

“Implicit constructor from an initializer list, making it possible to pass a brace-enclosed initializer list to a function expecting a `Span`. Such spans constructed from an initializer list must be of type `Span<const T>`.”

```
void Process(absl::Span<const int> x);  
Process({1, 2, 3});
```

— `absl/types/span.h`

3.1. This is a breaking change

Unfortunately, adding a `std::initializer_list` constructor is a breaking change.

Currently, when using `{ /* ... */ }` a constructor not taking `std::initializer_list` will be called. With this paper, depending on the arguments, the constructor taking `std::initializer_list` might be called.

3.1.1. Function overloads

A more problematic situation arises when considering function overloads.

```
#include <span>
#include <vector>

void f(std::span<const int>);

void f(const std::vector<int>&);

int main(){
    f({1,2,3});
}
```

As of today, this code is valid and calls `f(const std::vector<int>&)`. If this paper gets accepted, the function call would be ambiguous.

As `std::span` should be the vocabulary type for passing around a non-owning contiguous range of memory, overloading for both `std::span<const T>` and `const std::vector<T>&` should never be a necessity.

One would normally, as presented at the beginning of the paper and motivated in [the original span paper](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0122r1.pdf) (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0122r1.pdf>), replace `f(const std::vector<int>&)` with `f(std::span<const int>)`, and not have both overloads.

Thus as of today, one might be tempted to add those overloads because `span` does not have a `std::initializer_list` constructor, duplicating some part of its API without a good reason.

[P0921](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0921r2.pdf) (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0921r2.pdf>) states that the standard library reserves the right to add overloads, as this issue is not specific to `span`, but to any constructor of any class, and functions in general.

3.1.2. Unconstrained constructors

The proposed constructor for `std::initializer_list` is constrained for a span over constant elements. This reduces the breakage to a subset of types with an unconstrained constructor when creating a span, which are generally not common.

For those examples, the code behavior is unchanged as the constraint rules the new constructor out:

```
const std::vector<int>;

auto sp1 = std::span{v};
auto sp2 = std::span{v.begin(), v.end()};
auto begin = v.data();
auto end = begin + v.size();
auto sp3 = std::span{v.begin(), v.end()};
```

To use the proposed `std::initializer_list` constructor, one needs to spell the type out:

```
const std::vector<int>;

auto sp1 = std::span<const std::vector<int>>>{v};
```

Types with unconstrained constructors, like `void*` and `std::any`, are problematic:

```
void foo(span<void* const> s);

// without this paper, creates a span with one void*
// with this paper, creates a span of two void*
void* vp = nullptr;
span<void* const> sp{&vp, &vp+1};

// without this paper, creates a span with one std::any*
// with this paper, creates a span of two elements
std::any a;
span<std::any> sa{&a, &a+1};
```

As those types are more rare, and `std::span` is relatively new, the chance of this change to break existing code is low.

Also as `std::any a; std::vector<std::any> sa{&a, &a+1};` (or any other container) would create a vector of two elements, the proposed constructor removes an inconsistency that it is easy to overlook, and currently makes refactoring more difficult and error-prone.

3.1.3. Function signatures

One of the main use-cases for `span` is being used as a function parameter:

“*One of the major advantages of `span` over the common idiom of a “pointer plus length” pair of parameters is that it [...]*

— *span: bounds-safe views for sequences of objects*

For those intended use-cases the breaking change will be very uncommon as the type is normally spelled out.

For example, if a function signature would have been

```
void foo(const std::vector<int*>&);
```

when changing it to

```
void foo(std::span<int* const>);
```

then

```
int* ptr1 = ...;
int* ptr2 = ...;
foo({ptr1, ptr2});
```

can only be interpreted as a `std::span<int* const>` constructed with an `std::initializer_list<int*>`, and not a `std::span<int* const>` created from a pair of iterators.

If the type is not spelled out, as in

```
template <class T>
void foo(span<conts T> s){/* ... */}
```

then class template argument deduction generally will not work:

```
int* begin = nullptr;
int* end = nullptr;
foo({begin, end});
/*
error: no matching function for call to 'foo(<brace-enclosed initializer list>)'
foo({begin, end});
~~~~~^~~~~~
note: candidate: 'template<class T> void foo(std::span<const T>)'
void foo(std::span<const T> s){}
      ^~~
note:   template argument deduction/substitution failed:
note:   couldn't deduce template parameter 'T'
foo({begin, end});
~~~~~^~~~~~
*/
```

CPP

As the expected usage of `std::span` is to be used (A) as a parameter type and (B) with a non-deduced template argument, the breaking change (considering types like `std::span<void* const>`) should not affect much code.

4. Proposed Wording Changes

The following proposed wording changes against the working draft of the standard are relative to [N4892](http://open-std.org/jtc1/sc22/wg21/docs/papers/2021/n4892.pdf) (<http://open-std.org/jtc1/sc22/wg21/docs/papers/2021/n4892.pdf>).

In [version.syn], add:

```
#define __cpp_lib_span_init YYYYMMML // also in <span>
```

Apply following modifications to Header span synopsis [span.syn] :

```
#include <initializer_list>    // see [initializer.list.syn]

namespace std {
    // constants
    inline constexpr size_t dynamic_extent = numeric_limits<size_t>::max();
```

Apply following modifications to the definition of Class template span[views.span], Overview [span.overview] :

```

template<class ElementType, size_t Extent = dynamic_extent>class span {
public:// constants and types
using element_type = ElementType;
using value_type = remove_cv_t<ElementType>;
using size_type = size_t;
using difference_type = ptrdiff_t;
using pointer = element_type*;
using const_pointer = const element_type*;
using reference = element_type&;
using const_reference = const element_type&;
using iterator = implementation-defined; // see 22.7.3.7
using reverse_iterator = std::reverse_iterator<iterator>;
static constexpr size_type extent = Extent;

constexpr span() noexcept;
template<class It>
constexpr explicit(extent != dynamic_extent) span(It first, size_type count);
template<class It, class End>
constexpr explicit(extent != dynamic_extent) span(It first, End last);
template<size_t N>
constexpr span(type_identity_t<element_type> (&arr)[N]) noexcept;
template<class T, size_t N>
constexpr span(array<T, N>& arr) noexcept;
template<class T, size_t N>
constexpr span(const array<T, N>& arr) noexcept;
template<class R>
constexpr explicit(extent != dynamic_extent) span(R&& r);
constexpr explicit(extent != dynamic_extent) span(std::initializer_list<value_type> il) noexcept;
constexpr span(const span& other) noexcept = default;

```

Add the following text to Constructors, copy, and assignment [span.cons]

```
constexpr explicit(extent != dynamic_extent) span(std::initializer_list<value_type> il) noexcept;
```

Constrains: is_const_v<element_type> is true.

Preconditions: If extent is not equal to dynamic_extent, then il.size() is equal to extent.

Effects: Initializes data_ with il.begin() and size_ with il.size().

5. Acknowledgements

A big thank you to all those giving feedback for this paper.

Especially Arthur O'Dwyer, Barry Revzin, Jonathan Wakely, and Tomasz Kamiński for helping with the wording.